

CEG 4350/5350 – Operating Systems Internals and Design

Project 2 – Adding Keyboard Support

Learning Objectives

- Learn how to interface with the I/O ports in your OS.

Overview

In this project you will work with I/O ports in your OS. I/O ports can be accessed with port addresses in assembly. Some sample code is provided to help you so that you only have to write minimal assembly.

This project will use the progress you have made in “Project 1”. If you have not fully completed all the requirements of “Project 1”, Project 2 will be impossible.

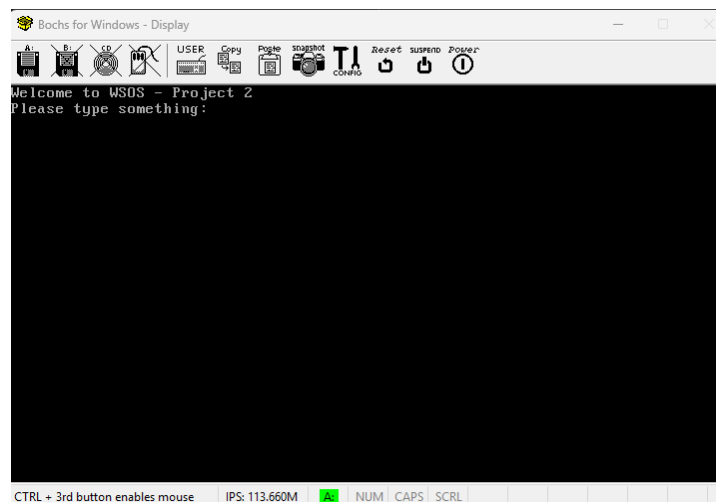
Reading From Keyboard

1. Modify project2.c and project2.h to implement 3 functions:

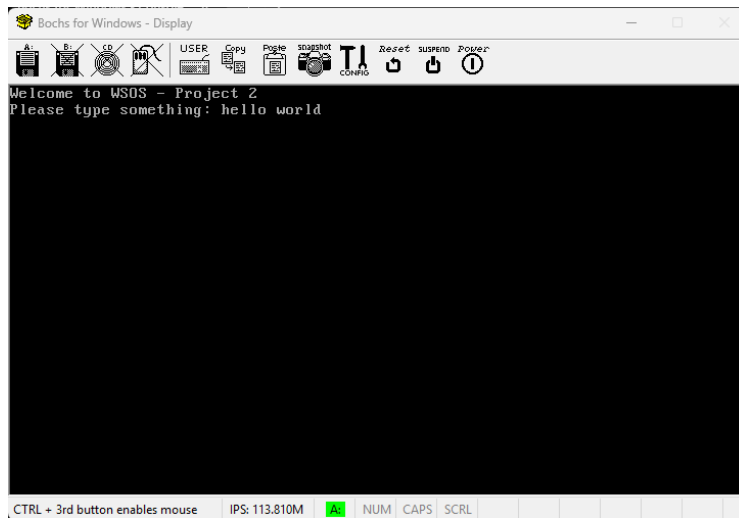
- a. volatile static char keymap[128] is a character array with characters ‘a’ through ‘z’, characters ‘0’ through ‘9’, as well as the ‘ ’ space character, ‘.’ character, ‘/’ character, and ‘\n’ enter key for newlines. Each index of the array should correspond to the scancode of the character. I.E. for the character ‘a’ you would have the following line of code: `[0x1E] = 'a'`,
 - i. Note: You do not have to account for characters other than the characters previously mentioned. Also, do not worry about capitalized characters or characters that require the SHIFT key to be pressed. Enabling the SHIFT key for capitalized and special characters is more difficult.
 - ii. Note: Only map the “key presses” not “key releases”.
- b. char getchar() must use polling (wait until keyboard status is ready) and when the keyboard status is “ready” retrieve the scancode from the keyboard. Once the scancode is retrieved, it must be converted to ASCII using the keyboard mapping.
 - i. returns the character typed on the keyboard
 - ii. Hint: Read the keyboard status port inside a while loop until the keyboard is ready
 - iii. Note: The keyboard will send unique scan codes for both pressing and releasing a key:
If a key is RELEASED the most significant bit (bit 7) of the scancode will be 1.
If a key is PRESSED the most significant bit (bit 7) of the scancode will be 0.

- c. void scan(char string[]) must populate the "char string[]" with the ASCII characters the user types in the keyboard until the user hits the enter key. After the user hits the enter key, you must terminate the string with a NULL terminator ('\0'). As the user is typing, each character should get printed to the display. You do not have to consider backspaces or hitting delete.
 - i. char string[] - the string to be filled with ASCII characters from all keyboard presses. Assume that the string argument will never exceed 100 characters and that the user will never type more than 100 characters.
 - ii. returns nothing
 - iii. Note: Do not store the user's enter key press ('\n') in the string. After the user has hit the enter key
- d. void scroll(int rows) must scroll video memory (i.e. xB8000 to xB8FA0) upwards by an amount of rows specified by the rows parameter. For example, when scroll(4) is called, all of the data in video memory addresses must be shifted down in memory by 640 addresses (4 rows * 80 screen positions per row * 2 memory addresses per screen position) which will make the screen appear to "scroll" upwards. After scrolling, you must clear the screen of the rows that existed beyond the scrolled row, otherwise the screen may retain stale text.
 - i. int rows – the number of rows to scroll by
 - ii. returns nothing
- e. int kernel() must prompt the user to enter a string. After the user enters their string, print the string back to them, then prompt them to enter again. Run this inside an infinite loop to allow the user to type as much as they want.
 - i. return 0

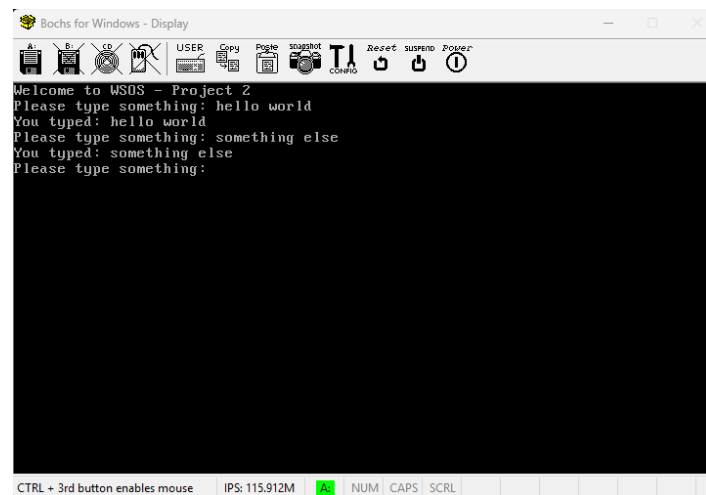
If you have implemented everything properly you should see the following results:



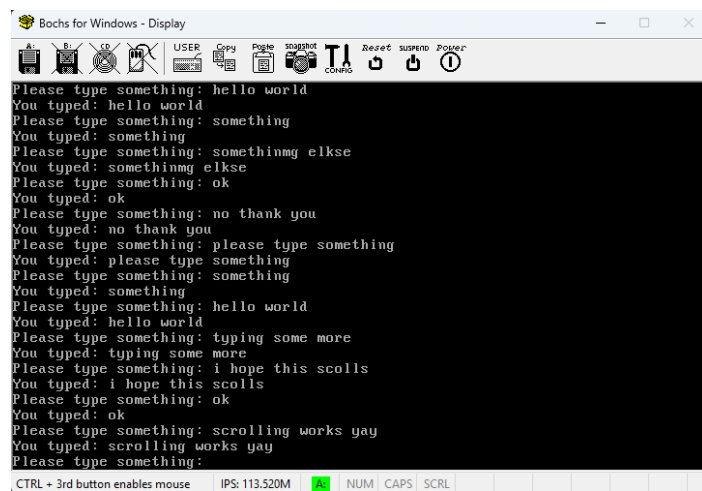
Beginning of OS



User types “hello world”



User continues to type



User types, scrolling the screen

How to Submit

To submit your code, commit your changes to your GitHub repository and include a link to your repository in the Pilot dropbox.

Grading

This lab is worth 5.00 points, distributed as follows:

Task	Points
OS compiles without errors (warnings are ok)	0.50
OS boots without errors (warnings are ok)	0.50
volatile static char keymap[128] maps 'a' - 'z', '0' - '1', ' ', and '\n'	1.00
getchar() is implemented	1.00
scan() is implemented	1.00
scroll() is implemented	1.00
Total	5.00